Coding Assignment 3: SLAM and Motion Planning

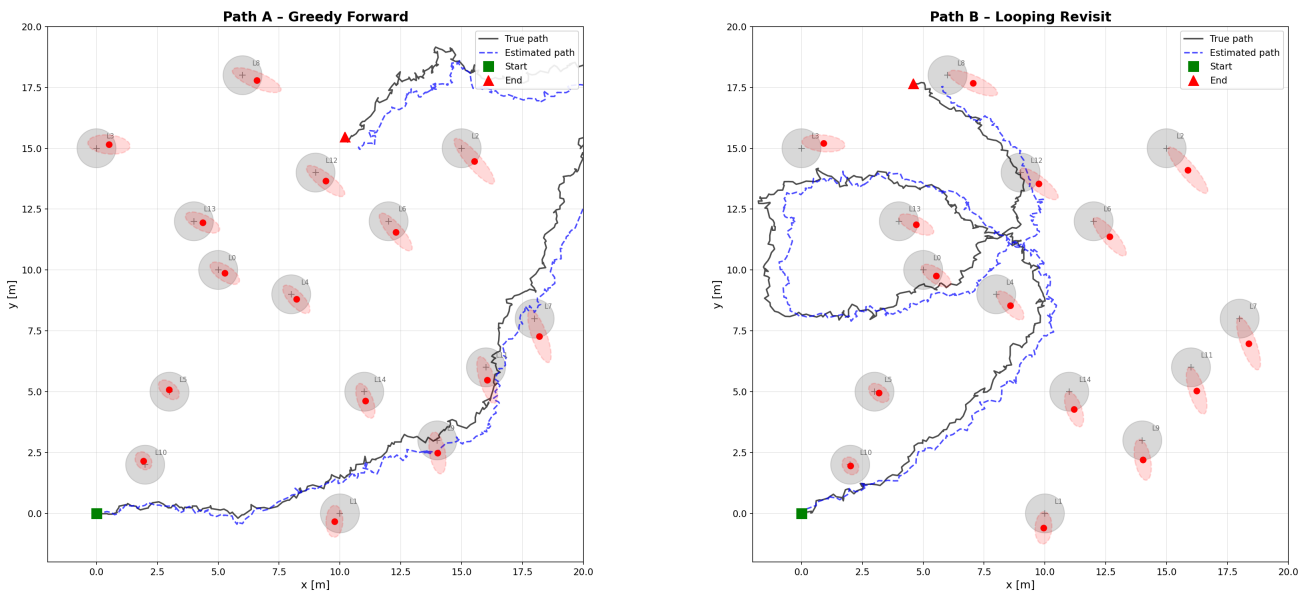
CS 188: Introduction to Robotics – Winter 2026

1 Part A: EKF-SLAM Trajectory and Uncertainty

1.1 A1. Path Design: Which Trajectory is Better for SLAM?

I tested two different control strategies in the same 15-landmark environment:

- **Path A (Greedy Forward):** L-shaped sweep. Straight east for 150 steps, turn left, straight north for 150 steps, turn left, then west. Each landmark is observed once or twice.
- **Path B (Looping Revisit):** Heads northeast, then enters tight circular orbits (ω=0.25 rad/s), translates, and does a second orbit. Landmarks near the orbit centres are observed 4–8 times.



(a) Path A: Greedy Forward. *(b)* Path B: Looping Revisit.

Figure 1: True path (black) vs. estimated path (blue dashed) with landmark 2σ ellipses. Green square = start, red triangle = end.

Metric	Greedy Forward	Looping Revisit
Final pose error	0.669 m	1.204 m
Mean pose error	0.837 m	0.775 m
Final pose uncertainty	0.572 m	0.607 m
Final avg landmark unc.	0.215	0.217
Landmarks observed	15/15	15/15

Table 1: A1 trajectory comparison (500 steps, medium noise).

Greedy Forward has a lower *final* pose error (0.669 vs. 1.204 m), but Looping has a lower *mean* error (0.775 vs. 0.837 m). This is because the looping path keeps re-observing landmarks, which triggers EKF corrections that pull the estimate back toward truth. The greedy path drifts during long straight segments but happens to end near landmarks that anchor the final estimate. Both paths observe all 15 landmarks, so final landmark uncertainties are nearly identical (≈ 0.215).

Uncertainty ellipse evolution. fig. 2 shows map snapshots at steps 50, 150, 250, 350, and 500. Each dot is an observed landmark. The colour encodes uncertainty: blue means confident, red means uncertain. The ellipse around each dot is the 2σ confidence region. A large ellipse means the EKF is unsure where the landmark is.

Under Path A, ellipses shrink when a landmark is first observed but stay the same size after that, because the robot moves on and never returns. Under Path B, landmarks near the orbit centre are re-observed on every loop. Each re-observation triggers an EKF update where the Kalman gain uses the cross-covariance to correct both the robot pose and landmark position. The ellipses visibly collapse from large red shapes (step 50) to tight blue dots (step 350). This is the **loop-closure effect**, the most important mechanism in EKF-SLAM for reducing uncertainty.

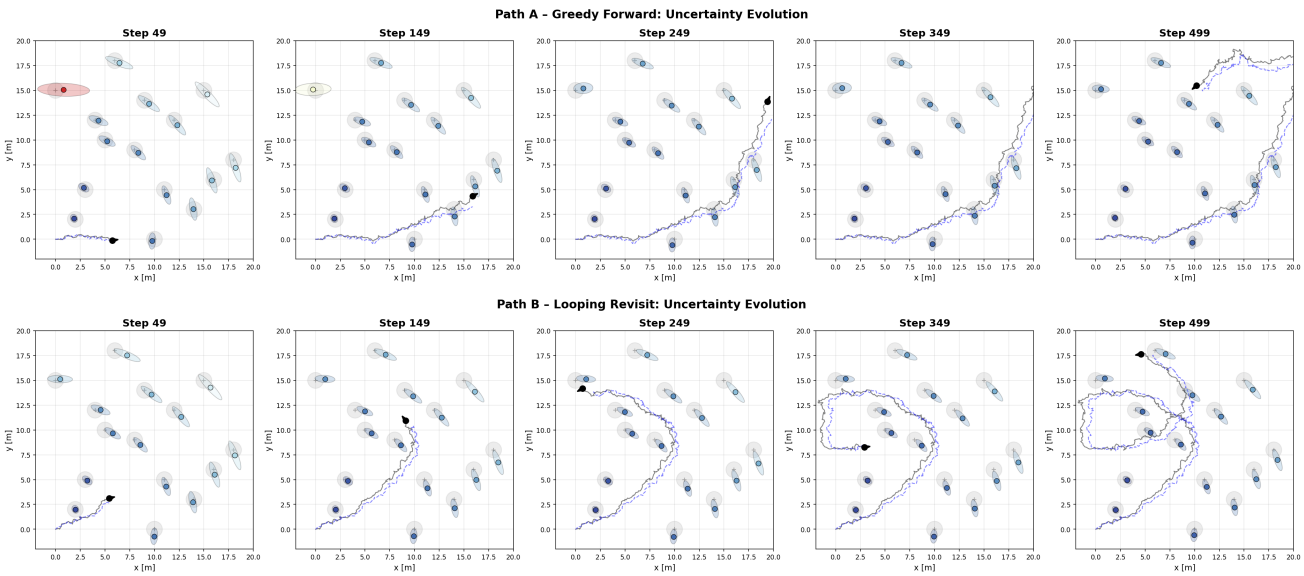


Figure 2: Uncertainty snapshots. *Top:* Path A, ellipses stay fixed after first observation. *Bottom:* Path B, ellipses collapse with repeated re-observation. Landmarks far from the orbit (e.g. top-right) keep larger ellipses.

1.2 A2. Noise Sensitivity

I used Path B for all noise experiments because it is the stronger baseline.

R Level	σ_{xy}	σ_θ	Pose Err	Q Level	σ_r	σ_b	Pose Err
Low	0.05	1.5°	0.278 m	Low	0.2	3.0°	1.263 m
Med	0.10	3.0°	1.204 m	Med	0.5	8.0°	1.204 m
High	0.30	8.0°	1.857 m	High	1.5	15.0°	1.351 m

Table 2: Noise sensitivity. Left: varying process noise $\boldsymbol{R}$ (Q fixed). Right: varying measurement noise $\boldsymbol{Q}$ (R fixed).

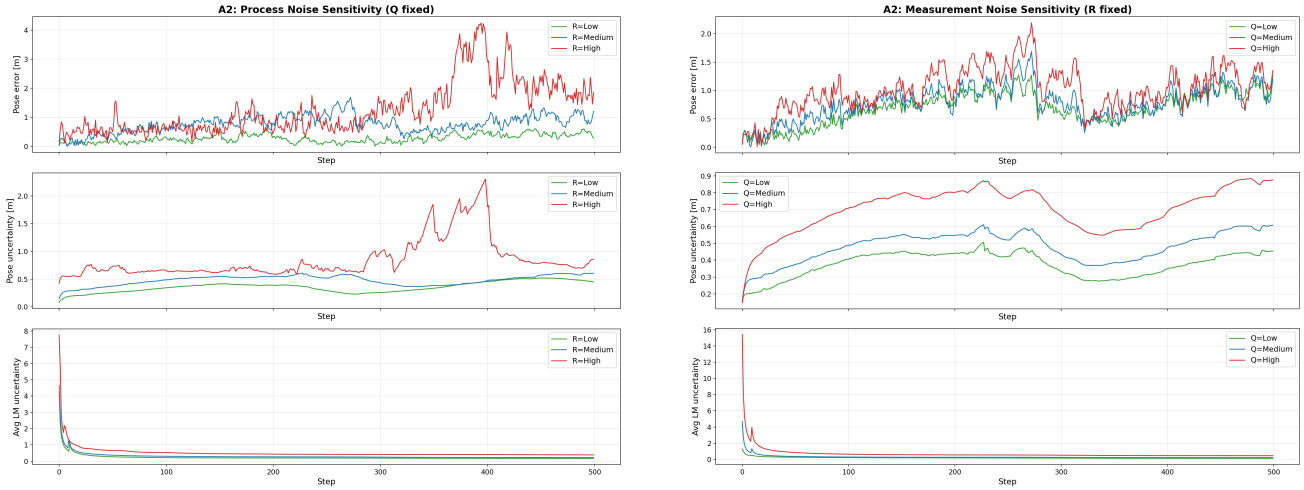
Process noise is far more damaging. Increasing R-std by 6× causes 6.7× more error, while increasing Q-std by 7.5× causes only 1.1× more error. Process noise corrupts the robot state at every predict step (500 total), so drift accumulates between observations. Measurement noise only degrades the quality of individual corrections and gets averaged out by repeated observations. Even under high measurement noise, the looping path stays at 1.351 m error. This confirms that revisiting landmarks gives built-in protection against sensor noise.

2 Part B: PRM vs RRT Planning Comparison

2.1 B1. Start/Goal Scenarios

- **Scenario 1 (Open):** Obstacle radius 0.8, start= $[1, 1]$, goal= $[19, 1]$. Wide gaps between obstacles.
- **Scenario 2 (Constrained):** Obstacle radius 1.0, start= $[1, 1]$, goal= $[17, 17]$. Tight passages.

Prediction: PRM should give shorter paths (A* on roadmap) but be slower. RRT may struggle with narrow passages but should be faster.



(a) Process noise sweep (***Q*** fixed).

(b) Measurement noise sweep (***R*** fixed).

Figure 3: A2 noise sensitivity. Higher process noise (left, red) is much worse across all metrics. Measurement noise (right) has a smaller effect.

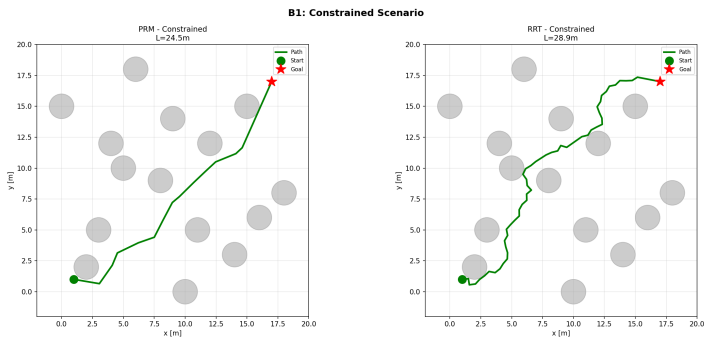


Figure 4: Constrained scenario. PRM (left) and RRT (right) with obstacle radius 1.0.

2.2 B2. Metrics Comparison (Constrained, N=30)

Both achieve 100% success. PRM produces 15% shorter paths because A* finds the optimal route on the roadmap. RRT is 26× faster because it does not build a full graph. The RRT paths zigzag visibly (fig. 4, right) because each extension steps toward a random sample. PRM's A* finds the shortest graph path through dense collision-free nodes. This is the core trade-off: PRM gives better paths but costs more time. RRT is fast but paths need post-processing.

2.3 B3. Parameter Studies

PRM: Going from 100 to 800 samples only shortens paths by 3.6%. The roadmap is dense enough at 200–400 nodes. More samples just add planning time.

RRT step_size: Minimal effect (28.40–28.98 m across 0.2–1.0). Smaller steps avoid obstacles better but need more iterations.

RRT goal_sample_rate: This is the most impactful parameter. Zero bias drops success to 95% because the tree explores randomly. A 0.7 bias yields 10% shorter paths by steering growth toward the goal. High bias works here because the direct route is mostly clear. In cluttered environments, high bias would waste iterations trying to extend through blocked areas.

3 Part C: Improving PRM and RRT

Improvement: path shortcutting. I added a post-processing step to both planners. For 200 iterations, randomly pick two non-adjacent waypoints i, j. If the straight line between them is collision-free, remove all waypoints in between. This targets the main weakness of both planners: PRM paths follow graph edges that may detour, and RRT paths have many small incremental steps.

Metric	PRM	RRT
Success rate	100%	100%
Avg path length	24.48 m	28.72 m
Avg time	1.762 s	0.068 s
Avg nodes	500	217

Table 3: B2 metrics (constrained scenario, 30 trials).

PRM: n_samples			RRT: goal_sample_rate		
`n_samples`	Succ.	Avg Len	`goal_rate`	Succ.	Avg Len
100	100%	25.17 m	0.0	95%	29.97 m
200	100%	24.85 m	0.1	100%	28.72 m
400	100%	24.57 m	0.3	100%	28.10 m
800	100%	24.27 m	0.7	100%	26.84 m

Table 4: B3 parameter studies. PRM: 8× more samples gives only 3.6% shorter paths. RRT: goal bias is the strongest factor.

Table 5: Shortcutting results (20 trials).

	Before	After	Len Δ	WP Δ
PRM	24.50 m	24.03 m	−2.0%	−69.5%
RRT	28.72 m	24.46 m	−14.8%	−90.9%

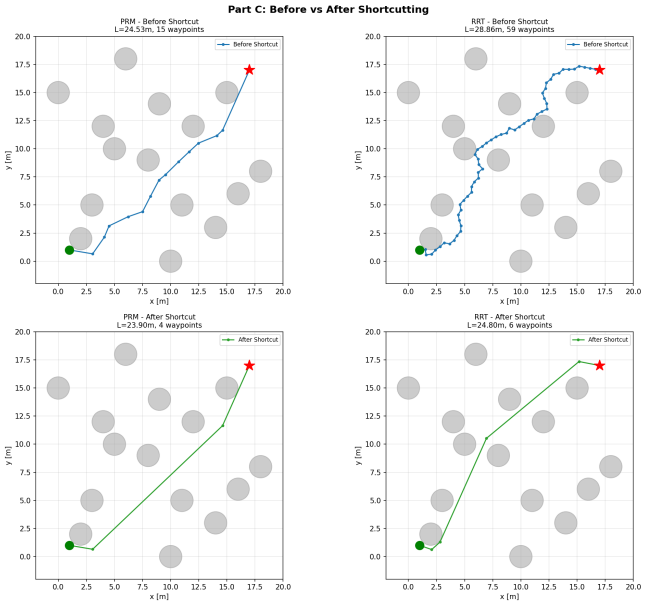


Figure 5: Before (blue) vs. after (green) shortcutting. RRT (right) shows large simplification.

RRT benefits the most: 14.8% shorter paths and 91% fewer waypoints, because its small incremental steps are easy to shortcut into long straight segments. PRM improves less in length (−2.0%) since A* already gives near-optimal paths, but still drops 70% of waypoints. After shortcutting, both planners produce paths around 24 m, which is likely close to the true shortest collision-free path. This means the planner choice matters less once post-processing is applied. In practice, RRT (26× faster) combined with shortcutting can match PRM's path quality at much lower cost.

Use of AI Tools

I used Claude Code to write experiment scripts and plotting code. I designed the experiments, verified outputs, and wrote all analysis myself.